COLEÇÃO IA DE VERDADE

IA de Verdade

Do primeiro prompt ao seu próprio servidor: agentes, automações, código e infraestrutura na prática

Volume 1

Entendendo e usando IA

```
$ ia --explicar "token"
> pedaço de texto que o modelo enxerga
$ ia --usar --no-dia-a-dia
> ok_
```

Giliarde Reis Pereira

IA de Verdade

Do primeiro prompt ao seu próprio servidor: agentes, automações, código e infraestrutura na prática
Volume 1 · Entendendo e usando IA

Giliarde Reis Pereira

IA de Verdade — Volume 1: Entendendo e usando IA
Copyright © 2026 Giliarde Reis Pereira. Todos os direitos reservados. Nenhuma parte desta obra pode ser reproduzida sem autorização do autor, exceto trechos curtos para citação com indicação da fonte.

Primeira edição: novembro de 2026. Data de referência das informações: 1º de novembro de 2026.

Marcas citadas pertencem a seus respectivos donos e aparecem apenas para identificar produtos e serviços. Skills de terceiros são indicadas por link e permanecem sob as licenças de seus autores; esta obra não reproduz o conteúdo delas.

As informações de finanças deste livro são educativas e não constituem recomendação de investimento. As instruções de impressão 3D e de hardware devem ser seguidas com os cuidados indicados pelos fabricantes.

Tipografia: Space Grotesk, Source Serif 4 e JetBrains Mono (SIL Open Font License).

CAPÍTULO 1

Bem-vindo à era dos agentes

Nível: Iniciante

Pré-requisitos: nenhum

Tempo estimado: 1 h de leitura + 30 min de projeto

O que você vai construir: um Diário de bordo de IA, o registro que você vai usar no livro inteiro

Objetivo do capítulo

Ao final deste capítulo você vai saber como a coleção está organizada, qual trilha de leitura combina com o seu perfil, como interpretar os selos e as fontes que aparecem no texto e o que precisa ter em mãos para acompanhar os projetos. Também vai montar o seu Diário de bordo de IA, um arquivo simples onde você registra o que pediu, o que a IA respondeu, quanto custou e o que você aprendeu.

Por que “era dos agentes”?

Até pouco tempo atrás, usar IA significava abrir uma janela de chat, digitar uma pergunta e ler a resposta. Isso continua existindo e continua útil. O que mudou é que os modelos de linguagem passaram a fazer coisas além de responder: eles leem arquivos, pesquisam na internet, preenchem planilhas, mandam mensagens, rodam comandos no seu computador e encadeiam várias dessas ações para cumprir um objetivo. Quando um sistema de IA recebe um objetivo, decide os passos, usa ferramentas e confere o resultado antes de parar, a gente chama esse sistema de **agente** (*agent*).

Essa mudança tem duas consequências práticas para você:

1. **O que dá para fazer ficou muito maior.** Um agente pode organizar seu extrato bancário, montar um calendário de posts ou gerar o arquivo 3D de uma peça.
2. **O que dá para dar errado também ficou maior.** Um chat que alucina te dá uma resposta errada; um agente que alucina pode apagar o arquivo errado ou mandar o e-mail errado.

Este livro existe para você aproveitar a primeira consequência sem cair na segunda. Por isso ele começa pelo básico (como a IA funciona, o que é cada peça) antes de te colocar para construir coisas.

> **PAPO DE PROFESSOR: A COZINHA E O CHEF**
>
> Pense num chat de IA como um chef que só pode **falar**: você pergunta como fazer um bolo e ele te dita a receita. Um agente é o mesmo chef com acesso à cozinha: ele abre a geladeira, liga o forno e tira o bolo pronto. É muito mais prático, mas agora faz diferença enorme se ele sabe onde fica o sal e se você deixou a faca de cozinha ao alcance dele. Boa parte deste livro é sobre **organizar a cozinha** antes de deixar o chef entrar.

Como a coleção está organizada

A coleção **IA de Verdade** tem cinco volumes e um volume de referência. Eles foram pensados para serem lidos em sequência, mas cada um se sustenta sozinho para quem já tem a base.

Volume	Tema	Para quem é o foco
1 (este)	Entendendo e usando IA: fundamentos, peças, prompt, contexto, custos, dia a dia, automações e bots sem código	Todo mundo
2	Construindo software com IA: programação assistida, APIs, Skills, RAG, MCP, avaliação, segurança	Quem programa ou quer programar
3	Agentes, automações e IA aplicada	Intermediário a avançado
4	Manuais do OpenClaw e do DSH, separados e trabalhando juntos	Quem quer um agente próprio

Volume	Tema	Para quem é o foco
5	Controle remoto e Home Lab	Quem quer rodar tudo em casa
Referência	Seleção comentada de Skills, combos, glossário	Consulta

Um projeto atravessa a coleção inteira como fio condutor: o **Assistente Casa**. Neste volume ele nasce como um bom conjunto de instruções e contexto (Capítulos 5 e 6). No Volume 2 vira um aplicativo com API e busca em documentos. No Volume 3 ganha agentes e automações. No Volume 4 passa a morar no OpenClaw, com o DSH como oficina. No Volume 5 fica hospedado num computador da sua casa, acessível do celular.

Os dois agentes da coleção: OpenClaw e DSH

Dois programas aparecem ao longo de todos os volumes, e é bom você conhecê-los pelo nome desde já:

- **OpenClaw** é um assistente de IA de código aberto (licença MIT) que roda no seu próprio computador e conversa com você pelos canais que você já usa, como Telegram, Discord, WhatsApp e Slack. A peça central dele se chama *Gateway*: um processo que fica ligado e coordena modelos, canais e ferramentas (github.com/openclaw/openclaw).
- **DSH (DeepSeek Harness)** é um *harness* de agentes de código aberto (MIT) desenvolvido pela DeepSeek, construído sobre a ideia de que "tudo é um plugin". Ele roda uma interface web local onde você trabalha com agentes. Na data de referência ele está em *developer preview*, e o próprio projeto avisa que haverá mudanças que quebram compatibilidade (github.com/deepseek-ai/deepseek-harness).

A ideia da coleção não é escolher um vencedor. Os dois têm perfis diferentes e se complementam: o OpenClaw é forte como assistente sempre ligado, presente nos seus canais de mensagem; o DSH é forte como bancada de trabalho extensível por plugins. No Volume 4 você vai aprender a operar cada um e a integrá-los. Aqui no Volume 1 eles aparecem como exemplos concretos dos conceitos, e você não precisa instalar nenhum dos dois para acompanhar.

Trilhas de leitura por perfil

Nem todo mundo chega ao livro com a mesma bagagem. Por isso existem três trilhas sugeridas. A Figura 1.1 mostra o caminho de cada perfil.

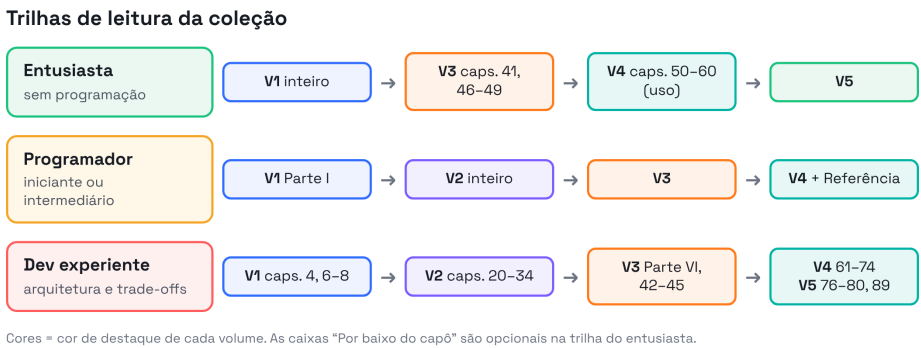


Figura 1.1 — Trilhas de leitura sugeridas para os três perfis de leitor.

Entusiasta sem programação. Você quer entender IA e usá-la de verdade no dia a dia, criar assistentes, bots e automações sem escrever código, e quem sabe montar um servidor em casa. Leia este volume inteiro, na ordem. Depois siga para os capítulos de uso do Volume 3 (assistentes, redes sociais, finanças e 3D avançados), os capítulos de uso dos manuais no Volume 4 e o Volume 5. Pode pular as caixas "Por baixo do capô" sem culpa; elas aprofundam, mas o texto principal não depende delas.

Programador iniciante ou intermediário. Você já escreve código e quer colocar IA nos seus projetos. Leia a Parte I deste volume com atenção (Capítulos 1 a 8), passe rápido pela Parte II escolhendo os temas que te interessam e mergulhe no Volume 2. Os exemplos em Python, TypeScript, Bash e PHP deste volume são pequenos de propósito: servem de aquecimento.

Dev experiente. Você quer arquitetura, trade-offs e implementação. Neste volume, os capítulos que mais vão te interessar são o 4 (as peças e as diferenças entre elas), o 6 (engenharia de contexto), o 7 (custos e delegação entre modelos) e o 8 (como projetar para trocar de fornecedor). As caixas "Por baixo do capô" foram escritas pensando em você.

Convenções do livro

Termos em português, com o original em inglês

O mundo da IA fala inglês, e muitas telas, documentações e mensagens de erro vão aparecer em inglês. Por isso, na primeira vez que um termo aparece, ele vem em português com o original entre parênteses: janela de contexto (*context window*), chamada de ferramenta (*tool calling*). Quando o termo em inglês é o que todo mundo usa no Brasil, como *prompt*, *token* e *Skill*, o livro usa o termo em inglês mesmo, sem forçar tradução.

Selos de maturidade: Experimental, Estável, Produção

Nem toda ferramenta ou técnica está no mesmo ponto de maturidade. Quando isso importa para a sua decisão, o texto usa três selos:

Selo	O que significa	Exemplo de postura
Experimental	Funciona, mas muda com frequência, pode quebrar entre versões e tem pouca documentação	Bom para aprender e testar; evite depender disso em algo importante
Estável	Tem versões estáveis, documentação oficial e mudanças anunciadas com antecedência	Bom para uso pessoal e projetos sérios, com plano de atualização
Produção	Estável e usado em escala, com política de suporte e de descontinuação publicada	Bom para coisas que outras pessoas dependem

O selo vale para a data de referência. O DSH, por exemplo, se declara *developer preview* em novembro de 2026, então ele entra como **Experimental** neste volume, por definição do próprio projeto. Isso não é crítica: é informação para você decidir onde usar.

Como o livro lê as fontes: confirmado, inferência e hipótese

Quando o assunto é IA, circula muita afirmação sem base. Para você saber o peso de cada informação, o texto deixa claro de onde ela vem:

- **Confirmado**: está escrito numa fonte oficial (documentação, página de preços, repositório do projeto, texto de lei), com link e data de consulta.

- **Inferência**: uma conclusão lógica a partir de fatos confirmados, apresentada como conclusão. Por exemplo: "como o preço é cobrado por token e a resposta longa usa mais tokens, pedir respostas curtas reduz o custo".
- **Hipótese**: algo plausível que ainda não tem confirmação, apresentado com "pode", "é possível que" ou "a tendência sugere". O livro usa hipóteses com moderação e nunca as transforma em fato.

Você vai usar essa mesma régua quando pedir para a IA pesquisar algo para você (Capítulo 9).

Como os projetos são apresentados

Todo projeto marcado com ★ segue a mesma estrutura, para você nunca ficar perdido:

1. **Problema**: o que estamos resolvendo.
2. **Resultado desejado**: como saber que deu certo.
3. **Arquitetura**: as peças e como elas se conectam.
4. **Ferramentas e pré-requisitos**.
5. **Instalação e configuração**.
6. **Implementação**: o passo a passo, com o código explicado.
7. **Execução e testes**.
8. **Resultado esperado**.
9. **Problemas comuns**.
10. **Segurança**.
11. **Melhorias**: para onde levar o projeto depois.

Em projetos com agentes, o texto sempre deixa explícito **o que o modelo decide** e **o que o sistema tradicional decide**. Essa separação é uma das ideias mais importantes da coleção, e você vai vê-la nascer no Capítulo 4.

IA não é mágica

Antes de seguir, vale fixar uma ideia que vai voltar muitas vezes: **IA não é solução mágica.** Ela é uma ferramenta muito poderosa para alguns tipos de problema e uma escolha ruim para outros.

Um modelo de linguagem é ótimo para lidar com linguagem: entender pedidos escritos de jeitos diferentes, resumir, reescrever, classificar textos, extrair informação de documentos bagunçados, gerar rascunhos. Ele é ruim (ou, no mínimo, caro e imprevisível) quando a tarefa exige exatidão absoluta e repetível, como somar valores de uma planilha, conferir se uma data é válida ou decidir se um pagamento foi aprovado. Para isso existem fórmulas, programas e regras fixas, que acertam sempre e custam quase nada.

Tarefa	Melhor com IA	Melhor sem IA
Resumir um contrato de 30 páginas	✓	
Somar os gastos do mês		✓ (planilha)
Classificar “PADARIA DO ZÉ 0412” como Alimentação	✓ (ou regra, se for repetitivo)	
Renomear 500 fotos pela data		✓ (script)
Escrever 5 opções de legenda para um post	✓	
Avisar quando o boleto vencer amanhã		✓ (agenda/ automação)

Repare na terceira linha: a mesma tarefa pode começar com IA e, depois que você entende o padrão, virar uma regra fixa. Esse movimento, da IA para o determinístico quando possível, é uma das formas mais eficientes de economizar dinheiro e ganhar confiabilidade. Você vai vê-lo na prática no Capítulo 11.

> **CUIDADO! RESPOSTAS CONVINCENTES NÃO SÃO RESPOSTAS CERTAS**
>
> Modelos de linguagem escrevem com segurança mesmo quando estão errados. Isso tem nome: **alucinação**, e o Capítulo 2 explica por que acontece. A regra prática desde já: para qualquer coisa que envolva dinheiro, saúde, direitos, segurança ou a reputação de alguém, confira a informação numa fonte primária antes de agir.

Kit inicial

Para acompanhar este volume você precisa de pouca coisa:

Obrigatório

- Um computador (Windows, macOS ou Linux) ou um celular com navegador.
- Uma conta em pelo menos um assistente de IA de uso geral. Existem planos gratuitos em vários provedores; o Capítulo 3 compara as opções e o Capítulo 9 ensina a configurar.
- Um editor de texto simples para o seu Diário de bordo (Bloco de Notas, TextEdit, VS Code ou o app de notas que você já usa).

Recomendado para os exemplos de código

- **Python 3.10 ou mais novo** (python.org).
- **Node.js** (versão LTS) se quiser rodar os exemplos em TypeScript (nodejs.org).
- **PHP 8.1 ou mais novo** se quiser rodar os exemplos em PHP (php.net).
- Um terminal: o do macOS ou Linux já serve; no Windows, o PowerShell ou o WSL.

Opcional

- Uma chave de API de algum provedor, para os exemplos que chamam modelos por código. Você só vai precisar disso a partir do Capítulo 7, e os exemplos foram escritos para funcionar com qualquer provedor compatível com o formato da OpenAI, incluindo a DeepSeek e modelos locais via Ollama.

- Uma impressora 3D, se quiser imprimir o projeto do Capítulo 13 (dá para fazer o projeto até o arquivo pronto sem impressora).

> **CUIDADO! CHAVE DE API É DINHEIRO**
>
> Uma chave de API funciona como um cartão de crédito: quem tiver a chave gasta na sua conta. Nunca cole a chave em prints, em repositórios públicos ou em conversas com a IA. Quando criar uma, defina um limite de gastos no painel do provedor. O Capítulo 17 volta a esse assunto.

★ Projeto: Diário de bordo de IA

Problema

Quem começa a usar IA costuma repetir os mesmos erros porque não lembra o que funcionou da última vez. Aquele prompt ótimo que você escreveu semana passada se perde no histórico do chat; aquela resposta errada que te fez perder uma hora não vira aprendizado.

Resultado desejado

Um arquivo único, simples e pesquisável, onde cada uso relevante de IA vira uma linha com: data, ferramenta e modelo, objetivo, o pedido (ou um resumo), a qualidade do resultado, o custo aproximado e uma lição. Você vai alimentar esse diário ao longo do livro e usá-lo nos Capítulos 5 (biblioteca de prompts), 7 (custos) e 8 (plano de saída de fornecedor).

Arquitetura

Nada de banco de dados nem app especial. Um arquivo **CSV** (valores separados por vírgula) abre em qualquer planilha (Excel, LibreOffice, Google Sheets) e em qualquer linguagem de programação.

Ferramentas e pré-requisitos

- Uma planilha ou um editor de texto.
- Opcional: Python 3.10+ para o script que gera um resumo do diário.

Implementação, versão sem código

Crie um arquivo chamado `diario_ia.csv` com esta primeira linha (o cabeçalho):

diario_ia.csv (cabeçalho)

```
data,ferramenta,modelo,objetivo,pedido,nota,custo_usd,licao
```

Cada coluna tem um papel:

Coluna	O que anotar	Exemplo
`data`	Data no formato AAAA-MM-DD	2026-11-03
`ferramenta`	Onde você usou	app do assistente, API, OpenClaw
`modelo`	O modelo, se aparecer	o nome que o app mostrar
`objetivo`	Para que era	resumir contrato de aluguel
`pedido`	Resumo do prompt	"resuma em 10 tópicos e liste multas"
`nota`	De 1 a 5, quanto a resposta resolveu	4
`custo_usd`	Custo, se souber (0 para plano gratuito)	0
`licao`	O que você faria diferente	pedir para citar a cláusula de cada tópico

Abra o arquivo na sua planilha e vá preenchendo. Duas ou três linhas por semana já valem a pena.

> **PAPO DE PROFESSOR: O CADERNO DO LABORATÓRIO**
>
> Cientista que não anota o experimento repete o experimento. O diário é o seu caderno de laboratório: ele transforma "acho que funcionou" em "funcionou nas 4 vezes que tentei, menos quando o PDF era escaneado".

Implementação, versão com código

Se você programa, um script curto lê o diário e mostra um resumo: quantos registros, nota média por ferramenta e custo total. O exemplo usa só a biblioteca padrão do Python.

cap01/resumo_diario.py

```python
import csv
import sys
from collections import defaultdict

arquivo = sys.argv[1] if len(sys.argv) > 1 else "diario_ia.csv"
notas = defaultdict(list)
custo_total = 0.0

with open(arquivo, newline="", encoding="utf-8") as f:
    for linha in csv.DictReader(f):
        notas[linha["ferramenta"]].append(int(linha["nota"]))
        custo_total += float(linha["custo_usd"] or 0)

total = sum(len(v) for v in notas.values())
print(f"Registros: {total}")
for ferramenta, lista in sorted(notas.items()):
    media = sum(lista) / len(lista)
    rotulo = f"  {ferramenta:<20} usos={len(lista):>3}"
    print(f"{rotulo}  nota média={media:.1f}")
print(f"Custo total: US$ {custo_total:.2f}")
```

Como funciona, linha a linha: `csv.DictReader` lê cada linha como um dicionário cujas chaves são os nomes do cabeçalho, por isso dá para escrever `linha["nota"]`. O `defaultdict(list)` cria automaticamente uma lista vazia para cada ferramenta nova, e o laço final calcula a média de cada uma. O `or 0` trata células de custo em branco como zero.

Para rodar:

```
python3 resumo_diario.py diario_ia.csv
```

Com um diário de exemplo de quatro linhas, a saída tem este formato:

```
Registros: 4
  API                  usos=  1  nota média=5.0
  app do assistente    usos=  3  nota média=3.7
Custo total: US$ 0.02
```

A mesma ideia em Bash, para quem só quer contar registros e somar custos direto no terminal (usa `awk`, presente no Linux e no macOS; não trata vírgulas dentro de campos entre aspas, então mantenha o diário sem vírgulas nos textos ou use o script em Python):

cap01/resumo_diario.sh

```
#!/usr/bin/env bash
# Uso: ./resumo_diario.sh diario_ia.csv
awk -F',' 'NR > 1 { n++; custo += $7 }
  END { printf "Registros: %d\nCusto total: US$ %.2f\n", n, custo }' "$1"
```

Testes

Crie um diário com três ou quatro linhas inventadas e rode o script. Confira de cabeça se a contagem e a soma batem. Depois apague as linhas de teste e comece o diário real.

Resultado esperado

Um arquivo `diario_ia.csv` com o cabeçalho e, ao fim da primeira semana de leitura, alguns registros reais.

Problemas comuns

- **A planilha juntou tudo numa coluna só.** Em português, muitas planilhas usam ponto e vírgula como separador. Na importação, escolha "vírgula" como separador, ou troque as vírgulas do arquivo por ponto e vírgula e ajuste o `-F','` do Bash para `-F';'` (no Python, use `csv.DictReader(f, delimiter=";")`).
- **Acentos estranhos (Ã§Ã£o).** O arquivo foi salvo em outra codificação. Salve como UTF-8.
- **Erro `KeyError: 'nota'` no Python.** O cabeçalho foi digitado diferente. Os nomes precisam ser exatamente os mesmos.

Segurança

O diário não deve conter senhas, chaves de API, dados bancários completos nem dados pessoais de outras pessoas. Anote "resumo do contrato de aluguel", não o contrato inteiro.

Melhorias

- Adicione uma coluna `tokens` quando chegar ao Capítulo 7 e começar a medir consumo.

- Use o diário como matéria-prima da sua biblioteca de prompts no Capítulo 5: os pedidos com nota 5 são candidatos naturais.
- Leitores que programam podem trocar o CSV por um banco SQLite quando passar de algumas centenas de linhas.

Erros comuns de quem está começando

1. **Tratar a IA como buscador.** Um modelo de linguagem não "procura" a resposta num banco de verdades; ele gera texto plausível. Quando a resposta depende de fatos atuais, use um assistente com busca na web e peça as fontes (Capítulo 9).
2. **Pedir pouco e esperar muito.** "Faz um post" gera algo genérico. Contexto e critérios geram algo útil (Capítulos 5 e 6).
3. **Colar dados sensíveis sem pensar.** Extrato completo, documento de identidade, dados de clientes (Capítulo 17).
4. **Achar que o plano gratuito de hoje é o de amanhã.** Planos mudam; tenha sempre uma alternativa (Capítulo 8).

Limitações deste volume

Este volume ensina a entender e usar IA, inclusive com um pouco de código. Ele não ensina a construir aplicações completas (Volume 2), a montar agentes sofisticados (Volume 3) nem a operar o OpenClaw e o DSH em profundidade (Volume 4). Quando um assunto passa do escopo, o texto indica onde ele continua.

Resumo

- Agentes são sistemas de IA que recebem um objetivo, decidem passos, usam ferramentas e conferem o resultado. Fazem muito mais que um chat e por isso exigem mais cuidado.
- A coleção tem cinco volumes e um de referência, com o Assistente Casa como fio condutor e o OpenClaw e o DSH como agentes de referência, usados juntos.

- Há três trilhas de leitura: entusiasta, programador iniciante/intermediário e dev experiente.
- O livro marca maturidade (Experimental, Estável, Produção) e o peso das informações (confirmado, inferência, hipótese).
- IA não é mágica: para exatidão repetível, soluções tradicionais costumam ser melhores.
- O Diário de bordo de IA é a sua ferramenta de aprendizado ao longo do livro.

CHECKPOINT

1. Qual a diferença entre um chat de IA e um agente, em uma frase?
2. Você precisa somar 200 valores de uma planilha. Usaria IA? Por quê?
3. O DSH entra como Experimental neste volume. De onde vem essa classificação?
4. O que nunca deve ir para o seu Diário de bordo?

Referências

- OpenClaw, repositório oficial: https://github.com/openclaw/openclaw (consulta em 2026-11-01).
- DeepSeek Harness (DSH), repositório oficial: https://github.com/deepseek-ai/deepseek-harness (consulta em 2026-11-01).
- DeepSeek Harness, página do produto: https://www.deepseek.com/en/harness/ (consulta em 2026-11-01).
- Python, módulo `csv`: https://docs.python.org/3/library/csv.html.

CAPÍTULO 2

Como a IA funciona, sem mistério

Nível: Iniciante

Pré-requisitos: Capítulo 1

Tempo estimado: 2 h de leitura + 30 min de projeto

O que você vai construir: um experimento com o mesmo pedido em 3 temperaturas, com e sem código

Objetivo do capítulo

Você vai entender, sem matemática pesada, o que acontece entre o momento em que digita uma pergunta e o momento em que a resposta aparece. Ao final, vai saber explicar o que são tokens, contexto, probabilidade e temperatura, por que modelos alucinam, o que muda num "modelo de raciocínio" e o que são embeddings. Essas ideias sustentam todo o resto do livro: quase todo problema prático com IA (resposta inventada, custo alto, instrução ignorada) tem a explicação neste capítulo.

O que é IA: de regras a aprendizado de máquina

Inteligência artificial é o nome guarda-chuva para sistemas que fazem tarefas que, em pessoas, associamos a inteligência: reconhecer imagens, entender linguagem, tomar decisões. Há duas grandes formas de construir esses sistemas.

Por regras. Alguém escreve as regras explicitamente: "se o e-mail contém 'ganhe dinheiro fácil', é spam". Funciona bem quando as regras são poucas e claras. Quebra quando o mundo é variado demais: o golpista escreve "g4nhe d1nheiro" e a regra não pega.

Por aprendizado de máquina (*machine learning*). Em vez de escrever as regras, você mostra milhares de exemplos ("isto é spam", "isto não é") e um algoritmo ajusta um modelo matemático até ele acertar a maioria. O modelo "aprende" padrões que ninguém escreveu à mão.

Os modelos de linguagem que você usa hoje são aprendizado de máquina levado a uma escala enorme. Mas repare: as regras não morreram. Num sistema bem feito, as duas coisas convivem. O modelo lida com a variedade da linguagem; as regras garantem o que precisa ser garantido.

> **PAPO DE PROFESSOR: ENSINAR UMA CRIANÇA A RECONHECER CACHORRO**
>
> Você não dá para uma criança a regra "quatro patas, focinho, rabo, late". Gato também tem quatro patas e rabo. Você aponta: "olha o cachorro", muitas vezes, cachorros grandes, pequenos, peludos, carecas. Um dia ela aponta para um cachorro de uma raça que nunca viu e acerta. Isso é aprendizado por exemplos. E, como a criança, o modelo às vezes chama um lobo de cachorro com toda a convicção.

Redes neurais numa analogia

O tipo de modelo por trás dos assistentes atuais é a **rede neural**. O nome vem de uma inspiração vaga no cérebro, mas a coisa em si é matemática: camadas e camadas de multiplicações e somas.

Imagine uma mesa de som gigantesca, com bilhões de botões. Cada botão é um número, chamado **peso** (*weight*) ou **parâmetro**. O texto entra de um lado, passa por todas as camadas de botões e sai do outro lado transformado em uma previsão. No começo os botões estão em posições aleatórias e a saída é lixo. O treinamento é o processo de girar cada botão um pouquinho, bilhões de vezes, na direção que deixa a saída menos errada.

Quando você lê que um modelo tem "27 bilhões de parâmetros", está lendo o número de botões dessa mesa. Mais parâmetros geralmente significam mais capacidade, e também mais memória e processamento para rodar. Esse número importa muito quando você quiser rodar um modelo no seu próprio computador (Capítulo 3 e Volume 3).

POR BAIXO DO CAPÔ: TRANSFORMERS E ATENÇÃO

A arquitetura dominante nos modelos de linguagem é o *Transformer*, apresentada em 2017 no artigo "Attention Is All You Need" (Vaswani et al., arxiv.org/abs/1706.03762). A peça central é o mecanismo de **atenção** (*attention*): para calcular a representação de cada token, o modelo pondera todos os outros tokens do contexto, decidindo "quanto olhar" para cada um. É isso que permite ao modelo ligar "ela" a "a impressora" três frases antes. O custo da atenção cresce com o tamanho do contexto, e essa é uma das razões técnicas por que contextos longos são mais caros e mais lentos. Variações como atenção esparsa e janelas deslizantes existem justamente para reduzir esse custo.

O que é um LLM

LLM (*Large Language Model*, modelo de linguagem grande) é uma rede neural treinada com uma quantidade enorme de texto para uma tarefa aparentemente simples: **prever o próximo pedaço de texto**. Só isso. Toda a capacidade de conversar, resumir, traduzir e programar emerge de ficar muito, muito bom nessa tarefa.

Pense no corretor do teclado do celular, que sugere a próxima palavra. Um LLM é esse corretor elevado a uma escala absurda: em vez de olhar as duas últimas palavras, ele olha centenas de milhares de tokens; em vez de um modelo pequeno treinado com suas mensagens, ele tem bilhões de parâmetros treinados com uma fração gigante do texto público da internet, livros, código e dados licenciados.

Para prever bem o próximo pedaço de um texto sobre física, o modelo precisa ter capturado algo de física. Para continuar bem um código, precisa ter capturado algo de programação. É por isso que "só prever o próximo token" produz algo que parece entender. Se isso é ou não "entender" de verdade é uma discussão filosófica interessante, mas para uso prático o que importa é saber **onde essa previsão é confiável e onde não é**.

Tokens: os pedaços que o modelo enxerga

O modelo não lê letras nem palavras. Ele lê **tokens**: pedaços de texto definidos por um **tokenizador** (*tokenizer*). Um token pode ser uma palavra inteira, um pedaço de palavra, um sinal de pontuação ou um espaço.

Para tornar isso concreto, veja como o tokenizador `o200k_base`, usado pela família GPT-4o da OpenAI e publicado na biblioteca de código aberto `tiktoken`, divide algumas frases:

Texto	Tokens	Como foi dividido
`Olá, tudo bem?`	5	`Olá` `,` `tudo` `bem` `?`
`Hello, how are you?`	6	`Hello` `,` `how` `are` `you` `?`
`inconstitucionalissimamente`	5	`in` `constit` `ucional` `issim` `amente`
`A impressora 3D travou na camada 42.`	13	`A` `impress` `ora` `3` `D` `trav` `ou` `na` `camada` `42` `.`

Três lições saem dessa tabela:

1. **Palavras comuns viram um token; palavras raras viram vários.** "camada" é um token só; "impressora" virou dois; a palavra gigante virou cinco.
2. **Números e espaços contam.** O "3D" virou três tokens (espaço, 3, D).
3. **Cada provedor tem o seu tokenizador.** O mesmo texto gera contagens diferentes em modelos diferentes. Como regra de bolso para português, conte algo entre 3 e 4 caracteres por token, e confira com a ferramenta do provedor quando precisar de precisão.

Tokens importam por dois motivos práticos: **o preço das APIs é cobrado por token** (Capítulo 7) e **o tamanho máximo de texto que o modelo aceita é medido em tokens**.

MÃO NA MASSA: CONTE TOKENS VOCÊ MESMO

Se você tem Python, instale a biblioteca e rode o script abaixo com qualquer frase:

```
pip install tiktoken
python3 tokens.py "Quanto custa imprimir uma peça de 40 gramas?"
```

cap02/tokens.py

```python
"""Conta tokens com o tokenizador o200k_base (pip install tiktoken)."""
import sys
import tiktoken

enc = tiktoken.get_encoding("o200k_base")
texto = " ".join(sys.argv[1:]) or "A impressora 3D travou na camada 42."
ids = enc.encode(texto)
print(f"{len(texto)} caracteres -> {len(ids)} tokens")
print(" | ".join(enc.decode([i]) for i in ids))
```

Na primeira execução a biblioteca baixa o arquivo do tokenizador, então precisa de internet. Sem programar, procure o "tokenizer" na documentação do seu provedor: vários oferecem uma página para colar texto e ver a contagem.

Contexto e janela de contexto

Contexto é tudo o que o modelo enxerga no momento de gerar a resposta: as instruções do sistema, o histórico da conversa, os arquivos que você anexou, os resultados de buscas e ferramentas, e a sua mensagem atual. O modelo não tem outra fonte de informação sobre a conversa. Se algo não está no contexto, para o modelo, não existe.

A **janela de contexto** (*context window*) é o tamanho máximo desse conjunto, medido em tokens, somando entrada e saída. Na data de referência, modelos de ponta de vários provedores anunciam janelas de 1 milhão de tokens; a DeepSeek, por exemplo, informa 1M de contexto e até 384K tokens de saída para seus modelos de API (api-docs.deepseek.com, consulta em 2026-11-01). Um milhão de tokens é algo como vários livros inteiros.

Janela grande não significa uso perfeito da janela. Três efeitos práticos:

- **Custo e lentidão.** Cada mensagem reenvia o contexto inteiro. Uma conversa longa fica mais cara e mais lenta a cada nova mensagem.
- **Atenção diluída.** Em contextos muito longos, informações no meio tendem a ser menos aproveitadas do que as do começo e do fim. Esse efeito foi documentado no estudo "Lost in the Middle" (Liu et al., arxiv.org/abs/2307.03172); modelos mais recentes melhoraram, mas o conselho prático continua valendo: coloque o que importa no começo ou no fim, e não entulhe o contexto.
- **Esquecimento aparente.** Quando a conversa passa do limite, o aplicativo precisa cortar ou resumir partes antigas. É por isso que o assistente "esquece" o que você disse lá no começo de uma conversa enorme.

O Capítulo 6 inteiro é sobre como montar e controlar o contexto, e o Capítulo 7 mostra o impacto disso na conta.

> **PAPO DE PROFESSOR: A MESA DE TRABALHO**
>
> O contexto é a mesa de trabalho do modelo. Ele só consegue usar o que está em cima da mesa. Uma mesa maior ajuda, mas uma mesa lotada de papel solto faz qualquer um perder o documento importante. Arrumar a mesa é mais eficiente do que comprar uma mesa maior.

Como a resposta é gerada: probabilidade e temperatura

Agora a parte mais importante do capítulo. A cada passo, o modelo calcula, para **cada token possível** do vocabulário (dezenas ou centenas de milhares), uma pontuação. Essas pontuações são convertidas em **probabilidades** que somam 100%. Depois, **um** token é escolhido, entra no texto, e o processo recomeça para o próximo. A resposta é construída token a token, sempre olhando para tudo o que veio antes.

A Figura 2.1 mostra um exemplo didático. O contexto é "O céu hoje está ___". Os valores são ilustrativos (não saíram de um modelo real), mas o cálculo das probabilidades é exatamente o que os modelos fazem.

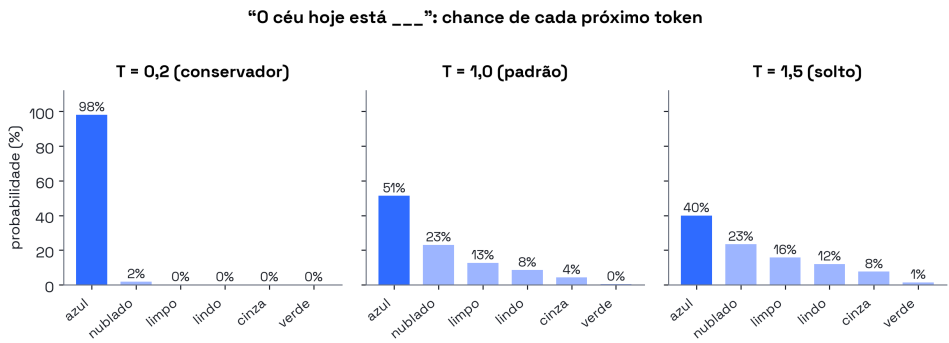


Figura 2.1 — Probabilidade de cada próximo token em três temperaturas. Valores de entrada ilustrativos; probabilidades calculadas com a função softmax.

A **temperatura** controla o quanto a escolha é "ousada":

- **Temperatura baixa** (perto de 0): o token mais provável domina. As respostas ficam previsíveis e repetíveis. Bom para extrair dados, classificar, responder com fatos do contexto, gerar código.
- **Temperatura média** (perto de 1): mantém a ordem, mas dá chance real a alternativas. É o padrão de muitas APIs.
- **Temperatura alta** (acima de 1): achata as diferenças. Surgem escolhas menos óbvias. Bom para brainstorming; ruim para precisão, porque aumenta a chance de sair do trilho.

Nem todo app deixa você mexer na temperatura, e alguns modelos de raciocínio a fixam ou ignoram. Em APIs, ela costuma ser um parâmetro chamado `temperature`. Há também o `top_p`, que limita o sorteio aos tokens mais prováveis até somar uma certa probabilidade. A recomendação comum das documentações é ajustar um ou outro, não os dois ao mesmo tempo.

POR BAIXO DO CAPÔ: SOFTMAX COM TEMPERATURA

As pontuações brutas se chamam *logits*. A probabilidade de cada token i é calculada pela função softmax com temperatura T: $p(i) = \exp(z_i/T) / \Sigma_j \exp(z_j/T)$. Dividir por T pequeno aumenta as diferenças entre os logits (o maior "ganha tudo"); dividir por T grande diminui. Com T tendendo a zero, a escolha vira *greedy* (sempre o mais provável). Na prática, mesmo com temperatura 0 a saída de uma API pode variar ligeiramente entre

> chamadas, por detalhes de paralelismo e arredondamento em GPU e pelo roteamento entre servidores. Não construa um sistema que dependa de saída idêntica byte a byte; dependa de validação (Volume 2).

★ Projeto: o mesmo pedido em 3 temperaturas

Problema

"Temperatura" parece um conceito abstrato até você ver o efeito. Queremos ver, com os próprios olhos, como ela muda as respostas, e entender em que tipo de tarefa usar cada faixa.

Resultado desejado

Três conjuntos de respostas para o mesmo pedido, um por temperatura, e uma anotação no seu Diário de bordo dizendo qual faixa funcionou melhor para qual tipo de tarefa.

Arquitetura

Duas versões. A **simulação** em código mostra o mecanismo com números controlados. O **experimento real** usa um modelo de verdade, por um playground do provedor (sem código) ou por API (com código, a partir do Capítulo 7).

Ferramentas e pré-requisitos

- Python 3.10+ (só biblioteca padrão) ou Node.js com TypeScript para a simulação.
- Para o experimento real: um playground ou console de API que exponha o parâmetro de temperatura. Vários provedores oferecem isso na própria plataforma de desenvolvedores, com cadastro.

Implementação 1: a simulação

O script abaixo usa os mesmos números da Figura 2.1. Ele calcula as probabilidades em três temperaturas e "sorteia" oito próximos tokens em cada uma. A semente fixa (`random.Random(42)`) faz o sorteio sair igual toda vez que você rodar, o que é ótimo para comparar.

cap02/temperatura.py

```
"""Simula a escolha do próximo token em 3 temperaturas.
Os 'logits' são valores ilustrativos, não saídas de um modelo real."""
import math
import random

LOGITS = {"azul": 4.0, "nublado": 3.2, "limpo": 2.6,
          "lindo": 2.2, "cinza": 1.5, "verde": -1.0}

def softmax(logits, temperatura):
    escalados = {t: v / temperatura for t, v in logits.items()}
    maior = max(escalados.values())          # estabilidade numérica
    exps = {t: math.exp(v - maior) for t, v in escalados.items()}
    soma = sum(exps.values())
    return {t: e / soma for t, e in exps.items()}

def sortear(probs, rng):
    return rng.choices(list(probs), weights=list(probs.values()))[0]

if __name__ == "__main__":
    rng = random.Random(42)        # semente fixa: resultado repetível
    for temp in (0.2, 1.0, 1.5):
        probs = softmax(LOGITS, temp)
        tabela = "  ".join(f"{t}={p:.0%}" for t, p in probs.items())
        frases = [sortear(probs, rng) for _ in range(8)]
        print(f"T={temp}: {tabela}")
        print(f"   8 sorteios: {' '.join(frases)}")
```

O que cada parte faz:

- `softmax` divide cada pontuação pela temperatura, aplica a exponencial e normaliza para somar 1. Subtrair o maior valor antes da exponencial não muda o resultado, mas evita números gigantes que estourariam a precisão do computador.
- `sortear` usa `random.choices` com pesos: tokens mais prováveis saem mais vezes, mas não sempre.
- O bloco final roda as três temperaturas e imprime a tabela e os sorteios.

Saída obtida com Python 3:

```
T=0.2: azul=98%  nublado=2%  limpo=0%  lindo=0%  cinza=0%  verde=0%
   8 sorteios: azul azul azul azul azul azul azul azul
T=1.0: azul=51%  nublado=23%  limpo=13%  lindo=8%  cinza=4%  verde=0%
   8 sorteios: azul azul azul azul azul azul nublado nublado
```

```
T=1.5: azul=40%  nublado=23%  limpo=16%  lindo=12%  cinza=8%  verde=1%
   8 sorteios: azul nublado lindo azul lindo limpo azul azul
```

Em T=0,2 o resultado é praticamente determinístico. Em T=1,5 aparecem "lindo" e "limpo", escolhas aceitáveis, e "verde" já tem 1% de chance: numa resposta de 500 tokens, pequenas chances de escolhas estranhas acumulam. É assim que temperatura alta aumenta o risco de a resposta sair do trilho.

Para quem prefere TypeScript, a mesma lógica (sem o sorteio):

cap02/temperatura.ts

```ts
const logits: Record<string, number> = {
  azul: 4.0, nublado: 3.2, limpo: 2.6, lindo: 2.2, cinza: 1.5,
  verde: -1.0,
};

function softmax(l: Record<string, number>, temp: number) {
  const esc = Object.entries(l).map(([t, v]) => [t, v / temp] as const);
  const maior = Math.max(...esc.map(([, v]) => v));
  const exps = esc.map(([t, v]) => [t, Math.exp(v - maior)] as const);
  const soma = exps.reduce((s, [, e]) => s + e, 0);
  return Object.fromEntries(exps.map(([t, e]) => [t, e / soma]));
}

for (const temp of [0.2, 1.0, 1.5]) {
  const p = softmax(logits, temp);
  const linha = Object.entries(p)
    .map(([t, v]) => `${t}=${Math.round(v * 100)}%`).join("  ");
  console.log(`T=${temp}: ${linha}`);
}
```

Rode com `npx tsx temperatura.ts`. A saída é a mesma tabela de probabilidades.

Implementação 2: o experimento real, sem código

1. Abra o playground ou console do provedor que você usa e escolha um modelo que aceite o parâmetro de temperatura.
2. Use este pedido, sem mudar uma vírgula entre as rodadas:

```
Crie um nome e um slogan de uma linha para uma pequena loja de
peças impressas em 3D no bairro. Responda só com "Nome: ..." e
"Slogan: ...".
```

1. Gere três respostas com temperatura 0,2; depois três com 1,0; depois três com a temperatura mais alta que o playground permitir (muitos vão até 2).
2. Repita com um pedido de **extração**: cole um parágrafo com datas e valores e peça "liste as datas e os valores em formato tabela". Gere três vezes em cada temperatura.

Testes

Compare as três respostas dentro de cada temperatura. Em temperatura baixa, elas devem ser idênticas ou quase. Em temperatura alta, devem variar bastante. No pedido de extração, verifique se alguma rodada em temperatura alta trouxe um valor errado ou inventado.

Resultado esperado

Uma linha no Diário de bordo para cada tipo de tarefa, por exemplo: "criativo: T=1,0 a 1,2 dá variedade sem bobagem; extração: T baixa, sempre". Os valores exatos dependem do modelo, então registre o modelo usado.

Problemas comuns

- **O playground não mostra temperatura.** Alguns modelos de raciocínio não aceitam o parâmetro ou o fixam. Escolha outro modelo para o experimento.
- **Mesmo em temperatura 0 as respostas mudaram um pouco.** Normal, como explicado na caixa "Por baixo do capô".
- **Erro ao rodar `tokens.py`.** A biblioteca precisa de internet para baixar o tokenizador na primeira vez.

Segurança

Use textos fictícios no experimento de extração. Playgrounds de desenvolvedor costumam cobrar por uso: confira se há limite de gastos configurado.

Melhorias

No Capítulo 7 você vai rodar este mesmo experimento por API, com código, e medir quantos tokens cada rodada consumiu.

Treinamento × inferência

Dois momentos muito diferentes da vida de um modelo costumam ser confundidos. A Figura 2.2 coloca os dois lado a lado.

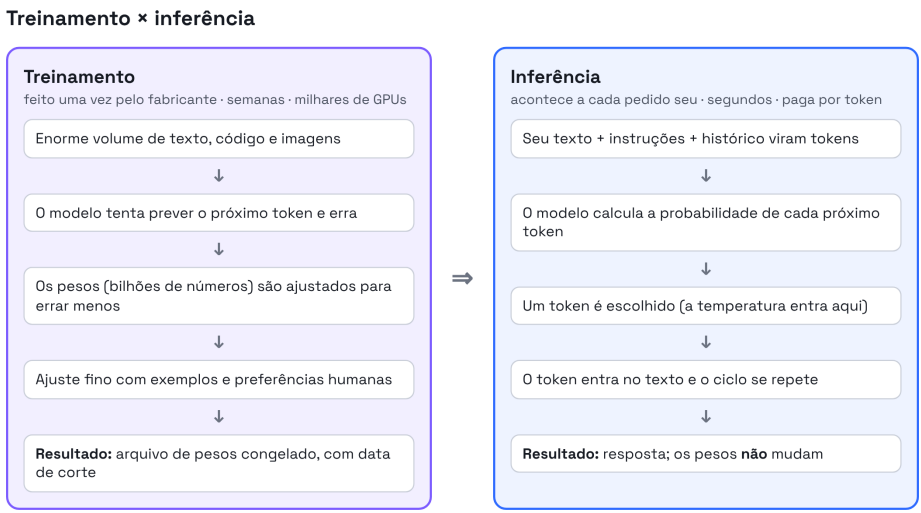


Figura 2.2 — Treinamento acontece uma vez, no fabricante; inferência acontece a cada pedido seu e não altera o modelo.

Treinamento é quando o fabricante ajusta os pesos. Acontece antes de o modelo chegar até você, custa uma fortuna e termina numa **data de corte de conhecimento** (*knowledge cutoff*): o modelo não sabe nada do que aconteceu depois dela, a não ser que essa informação chegue pelo contexto (por exemplo, por uma busca na web). Depois do treinamento bá-

sico vem o **ajuste fino** (*fine-tuning*) com exemplos de conversas e com preferências humanas, que transforma um "completador de texto" num assistente que segue instruções.

Inferência é o uso: cada vez que você manda uma mensagem, o modelo roda e gera a resposta. Os pesos não mudam. Isso derruba um mito comum:

> **CUIDADO! O MODELO NÃO APRENDE COM A SUA CONVERSA EM TEMPO REAL**
>
> Quando você corrige o assistente e ele "aprende", o que aconteceu foi que a correção entrou no contexto da conversa. Na próxima conversa, sem aquela correção no contexto, o erro volta. Recursos de **memória** dos apps (Capítulo 9) funcionam guardando anotações e colocando-as de volta no contexto, não alterando o modelo. Separadamente, alguns provedores podem usar conversas para treinar modelos futuros, conforme a política e as configurações da sua conta. Isso é uma questão de privacidade, tratada no Capítulo 17, e não tem nada a ver com o modelo melhorar durante a sua conversa.

Capacidades e limitações

Juntando tudo o que vimos, dá para prever onde um LLM vai bem e onde tropeça.

Vai bem	Tropeça
Entender pedidos escritos de muitos jeitos	Contas longas e exatas sem ferramenta
Resumir, reescrever, traduzir, mudar o tom	Fatos posteriores à data de corte
Extrair dados de texto bagunçado	Citar fontes exatas de memória
Classificar textos em categorias	Contar letras e caracteres (vê tokens, não letras)
Gerar rascunhos, ideias, variações	Manter consistência perfeita em textos muito longos
Explicar código e conceitos	Saber quando não sabe
Seguir formatos (tabela, JSON, lista)	Garantir que o formato sai certo 100% das vezes

Repare que vários itens da coluna da direita são resolvidos dando **ferramentas** ao modelo: uma calculadora para as contas, uma busca para os fatos recentes, um validador para o formato. Essa é a ponte para o Capítulo 4.

Alucinações

Alucinação (*hallucination*) é quando o modelo gera algo plausível, bem escrito e falso: um artigo científico que não existe, uma função que a biblioteca não tem, uma cláusula que não está no contrato.

Com o que você já sabe, a causa fica clara. O modelo escolhe tokens prováveis, não tokens verdadeiros. Quando o contexto e o treinamento dão base sólida, provável e verdadeiro coincidem. Quando não dão, o modelo continua produzindo o texto mais provável para aquela situação, e o texto mais provável para "o artigo que comprova isso é..." é algo com cara de artigo, com autores e ano. Os próprios processos de treinamento e avaliação, ao recompensar respostas confiantes, podem incentivar o chute no lugar do "não sei".

Como reduzir:

1. **Dê a fonte no contexto.** "Responda só com base no documento abaixo; se a resposta não estiver nele, diga que não está."
2. **Peça a citação.** "Para cada afirmação, cite o trecho do documento."
3. **Use busca com fontes** para fatos recentes, e abra as fontes (Capítulo 9).
4. **Baixe a temperatura** em tarefas factuais.
5. **Valide o que dá para validar automaticamente**: o código compila? O link abre? A soma bate?
6. **Desconfie de precisão demais**: números redondos, nomes completos e datas exatas que você não forneceu merecem conferência.

Nenhuma dessas medidas zera o risco. Elas o reduzem a um nível compatível com o uso, e o nível aceitável depende do que está em jogo.

Multimodalidade

Modelos **multimodais** trabalham com mais de um tipo de dado: texto, imagem, áudio, vídeo. A ideia por baixo é a mesma: tudo vira tokens. Uma imagem é dividida em pedaços (*patches*) que viram tokens de imagem; um áudio vira tokens de áudio. Por isso imagens e áudios também consomem a janela de contexto e são cobrados.

Na prática, multimodalidade permite: fotografar uma peça 3D com defeito e perguntar o que deu errado (Capítulo 13); mandar um áudio e receber texto; pedir uma imagem para um post (Capítulo 12). As mesmas limitações valem: o modelo pode "ver" algo que não está na foto, e vale conferir.

Modelos de raciocínio

Alguns modelos são treinados para **pensar antes de responder**: eles geram uma sequência de raciocínio intermediário (às vezes visível, às vezes oculta ou resumida) antes da resposta final. São chamados de **modelos de raciocínio** (*reasoning models*), e muitos provedores oferecem isso como um modo que você liga e desliga. Na DeepSeek, por exemplo, o modo de raciocínio é ativado por um parâmetro na requisição, e os antigos nomes de modelo separados para chat e raciocínio foram aposentados em 24 de julho de 2026 (api-docs.deepseek.com, consulta em 2026-11-01).

Vantagens: vão melhor em matemática, lógica, planejamento e código com várias etapas. Desvantagens: são mais lentos e consomem mais tokens, porque o raciocínio também é texto gerado e, em geral, é **cobrado como saída**. Para "reescreva este e-mail num tom mais formal", raciocínio é desperdício. Para "planeje a migração desta planilha para um sistema novo considerando estas 12 restrições", costuma valer.

Embeddings em linguagem de gente

Por último, um conceito que vai aparecer muito nos Volumes 2 e 3. Um **embedding** é uma lista de números (um vetor) que representa o **significado** de um texto. Textos com significados parecidos geram vetores parecidos, mesmo usando palavras diferentes.

> **PAPO DE PROFESSOR: O MAPA DAS IDEIAS**
>
> Imagine um mapa onde cada frase é um ponto. “Meu cartão foi clonado” e “fizeram compras no meu cartão que não reconheço” ficam vizinhas, mesmo sem nenhuma palavra em comum além de “cartão”. “Receita de bolo de cenoura” fica do outro lado do mapa. Um embedding é o endereço de uma frase nesse mapa, só que com centenas ou milhares de coordenadas em vez de duas.

Para que serve: **busca por significado** (achar o trecho do manual que responde à pergunta, mesmo com outras palavras), **agrupar** textos parecidos e **recomendar** conteúdo similar. É a base do RAG (*Retrieval-Augmented Generation*), a técnica de buscar documentos relevantes e colocá-los no contexto antes de responder, que o Capítulo 15 usa de forma simples e o Volume 2 aprofunda.

Erros comuns

1. **Achar que o modelo “sabe” coisas atuais.** Sem busca, ele só sabe o que estava no treinamento.
2. **Achar que corrigiu o modelo para sempre.** A correção vive no contexto daquela conversa.
3. **Usar temperatura alta para tarefas factuais.**
4. **Colar documentos enormes “por garantia”.** Mais contexto custa mais e pode piorar a resposta.
5. **Pedir contagem de letras ou contas longas sem ferramenta** e confiar no resultado.

Limitações deste capítulo

As explicações aqui são simplificações fiéis, não descrições completas. Modelos reais têm etapas adicionais (normalização, amostragem com várias técnicas combinadas, filtros de segurança) que variam por provedor e nem sempre são públicas.

Boas práticas

- Coloque no contexto a informação que o modelo precisa; não confie na memória do treinamento para fatos importantes.
- Escolha a temperatura pela tarefa: baixa para precisão, média para escrita, alta só para gerar ideias.
- Ligue o modo de raciocínio para problemas de várias etapas, e desligue para tarefas simples.
- Peça citações quando a resposta vier de documentos.

Resumo

- LLMs são redes neurais treinadas para prever o próximo token; as demais capacidades emergem disso.
- Tokens são os pedaços de texto que o modelo enxerga; preço e limites são medidos neles.
- Contexto é tudo o que o modelo vê; a janela de contexto é o limite. Contexto organizado vale mais que contexto grande.
- A resposta é gerada token a token, por probabilidade; a temperatura controla a ousadia da escolha.
- Treinamento muda o modelo; inferência (o seu uso) não muda.
- Alucinação é consequência natural da geração por probabilidade; reduz-se com fontes no contexto, citações, busca e validação.
- Modelos de raciocínio pensam antes de responder: melhores em problemas complexos, mais lentos e caros.
- Embeddings representam significado como números e permitem busca por sentido.

CHECKPOINT

1. Por que “inconstitucionalissimamente” custa mais tokens do que “casa”?
2. Você vai extrair valores de notas fiscais. Que temperatura usaria e por quê?
3. Você corrigiu o assistente ontem e hoje ele repetiu o erro. O que aconteceu?
4. Dê um exemplo de tarefa em que o modo de raciocínio é desperdício.
5. O que um embedding permite fazer que uma busca por palavra-chave não permite?

Referências

- Vaswani, A. et al. “Attention Is All You Need”, 2017. https://arxiv.org/abs/1706.03762.
- Liu, N. F. et al. “Lost in the Middle: How Language Models Use Long Contexts”, 2023. https://arxiv.org/abs/2307.03172.
- OpenAI, biblioteca `tiktoken` (tokenizador `o200k_base`), versão 0.14.0. https://github.com/openai/tiktoken.
- DeepSeek API Docs, modelos e preços (contexto, saída máxima, modo de raciocínio). https://api-docs.deepseek.com/quick_start/pricing (consulta em 2026-11-01).